Importing Libraries

```
import time
import datetime
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder
from sklearn.metrics import r2_score,mean_absolute_error,mean_squared_er
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_regression
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error,mean_squared_error,root_
from sklearn.svm import SVR
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.tree import ExtraTreeRegressor
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.linear_model import SGDRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.dummy import DummyRegressor
from sklearn.model_selection import cross_val_score
from warnings import filterwarnings
filterwarnings("ignore")
```

DATA COLLECTION AND PREPROCESSING

```
#LOADING THE DATASET (HEAD)
df=pd.read_csv("/content/cleaned_dataset.csv")
print(df.head())
```

```
     Location  Year  Kilometers_Driven Fuel_Type  ...   Power Seats  Pri
0      Mumbai  2010              72000       CNG  ...   58.16   5.0   1.
1        Pune  2015              41000    Diesel  ...  126.20   5.0  12.
2     Chennai  2011              46000    Petrol  ...   88.70   5.0   4.
3     Chennai  2012              87000    Diesel  ...   88.76   7.0   6.
4  Coimbatore  2013              40670    Diesel  ...  140.80   5.0  17.

[5 rows x 12 columns]
```

```
#LOADING THE DATASET (TAIL)
print(df.tail())
```

```
        Location  Year  Kilometers_Driven  ... Seats Price      Brand
6014       Delhi  2014              27365  ...   5.0  4.75     Maruti
6015      Jaipur  2015             100000  ...   5.0  4.00    Hyundai
6016      Jaipur  2012              55000  ...   8.0  2.90   Mahindra
6017     Kolkata  2013              46000  ...   5.0  2.65     Maruti
6018  Hyderabad  2011              47000  ...   5.0  2.50  Chevrolet

[5 rows x 12 columns]
```

```
#CHECKING THE SIZE OF DATASET
print(df.shape)
```

```
(6019, 12)
```

```
#CHECKING THE INFORMATION OF DATASET
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Location           6019 non-null   object 
 1   Year               6019 non-null   int64  
 2   Kilometers_Driven  6019 non-null   int64  
 3   Fuel_Type          6019 non-null   object 
 4   Transmission       6019 non-null   object 
 5   Owner_Type         6019 non-null   object 
 6   Mileage            6017 non-null   float64
 7   Engine             5983 non-null   float64
 8   Power              5876 non-null   float64
 9   Seats              5977 non-null   float64
 10  Price              6019 non-null   float64
 11  Brand              6019 non-null   object 
dtypes: float64(5), int64(2), object(5)
memory usage: 564.4+ KB
None
```

```
#CHECKING THE NULL VALUES IN DATASET
print(df.isnull().sum())
```

```
Location              0
Year                  0
Kilometers_Driven     0
Fuel_Type             0
Transmission          0
Owner_Type            0
Mileage               2
Engine               36
Power               143
Seats                42
Price                 0
Brand                 0
dtype: int64
```

```
#FILLING THE NULL VALUES
df['Power'] = df['Power'].fillna(df['Power'].median())
```

```
df['Seats'] = df['Seats'].fillna(df['Seats'].median())
```

```
df['Engine'] = df['Engine'].fillna(df['Engine'].median())
```

```
df['Mileage'] = df['Mileage'].fillna(df['Mileage'].median())
```

```
#VERIFYING IF THE DATASET HAS ANY NULL VALUES
print(df.isnull().sum())
```

```
Location             0
Year                 0
Kilometers_Driven    0
Fuel_Type            0
Transmission         0
Owner_Type           0
Mileage              0
Engine               0
Power                0
Seats                0
Price                0
Brand                0
dtype: int64
```

```
#VERIFYING SOME STATISCAL MEASURES
print(df.describe())
```

```
              Year  Kilometers_Driven  ...        Seats        Price
count  6019.000000       6.019000e+03  ...  6019.000000  6019.000000
mean   2013.358199       5.873838e+04  ...     5.276790     9.479468
std       3.269742       9.126884e+04  ...     0.806346    11.187917
min    1998.000000       1.710000e+02  ...     0.000000     0.440000
25%    2011.000000       3.400000e+04  ...     5.000000     3.500000
50%    2014.000000       5.300000e+04  ...     5.000000     5.640000
75%    2016.000000       7.300000e+04  ...     5.000000     9.950000
max    2019.000000       6.500000e+06  ...    10.000000   160.000000

[8 rows x 7 columns]
```

```
#PRINTING THE COLUMNS
print(df.columns)
```

```
Index(['Location', 'Year', 'Kilometers_Driven', 'Fuel_Type', 'Transmissi
       'Owner_Type', 'Mileage', 'Engine', 'Power', 'Seats', 'Price', 'Br
      dtype='object')
```

FEATURE ENGINEERING

```
print(df["Location"].unique())
```

```
['Mumbai' 'Pune' 'Chennai' 'Coimbatore' 'Hyderabad' 'Jaipur' 'Kochi'
 'Kolkata' 'Delhi' 'Bangalore' 'Ahmedabad']
```

```
ordinal_encoder=OrdinalEncoder()
df['Location']=ordinal_encoder.fit_transform(df[['Location']])
print(df['Location'].unique())
```

```
[ 9. 10.  2.  3.  5.  6.  7.  8.  4.  1.  0.]
```

```
print(df["Fuel Type"].unique())
```

```
['CNG' 'Diesel' 'Petrol' 'LPG' 'Electric']
```

```
ordinal_encoder=OrdinalEncoder()
df['Fuel_Type']=ordinal_encoder.fit_transform(df[['Fuel_Type']])
print(df['Fuel Type'].unique())
```

```
[0. 1. 4. 3. 2.]
```

```
print(df["Transmission"].unique())
```

```
['Manual' 'Automatic']
```

```
ordinal_encoder=OrdinalEncoder()
df['Transmission']=ordinal_encoder.fit_transform(df[['Transmission']])
print(df['Transmission'].unique())
```

```
[1. 0.]
```

```
print(df['Owner Type'].unique())
```

```
['First' 'Second' 'Fourth & Above' 'Third']
```

```
ordinal_encoder=OrdinalEncoder()
df['Owner_Type']=ordinal_encoder.fit_transform(df[['Owner_Type']])
print(df['Owner_Type'].unique())
```

```
[0. 2. 1. 3.]
```

```
print(df['Brand'].unique())
```

```
['Maruti' 'Hyundai' 'Honda' 'Audi' 'Nissan' 'Toyota' 'Volkswagen' 'Tata'
 'Land' 'Mitsubishi' 'Renault' 'Mercedes-Benz' 'BMW' 'Mahindra' 'Ford'
 'Porsche' 'Datsun' 'Jaguar' 'Volvo' 'Chevrolet' 'Skoda' 'Mini' 'Fiat'
 'Jeep' 'Smart' 'Ambassador' 'Isuzu' 'ISUZU' 'Force' 'Bentley'
 'Lamborghini']
```

```
ordinal_encoder=OrdinalEncoder()
df['Brand']=ordinal_encoder.fit_transform(df[['Brand']])
print(df['Brand'].unique())
```

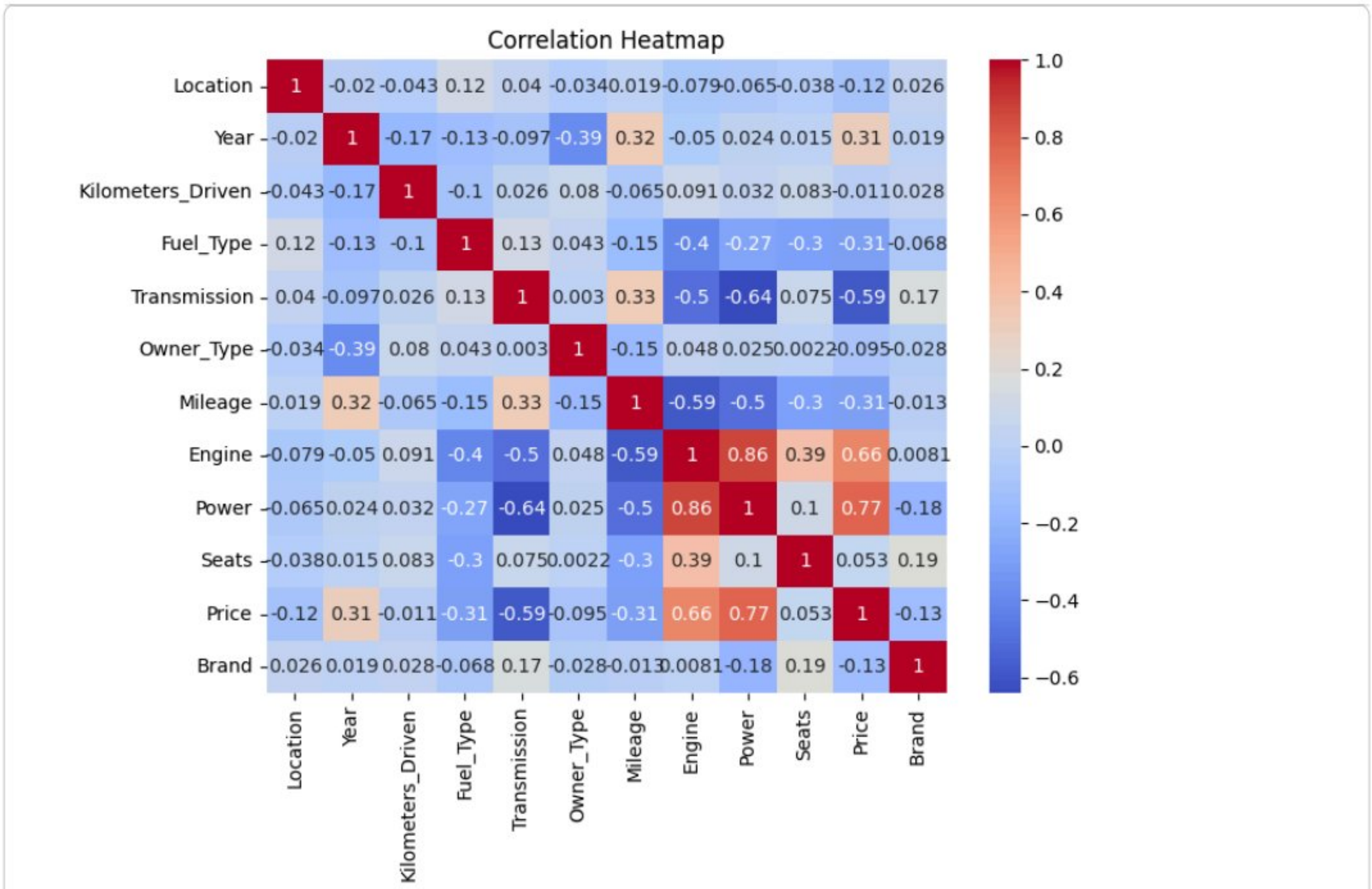
```
[18. 10.  9.  1. 22. 28. 29. 27. 16. 21. 24. 19.  2. 17.  8. 23.  5. 13.
 30.  4. 25. 20.  6. 14. 26.  0. 12. 11.  7.  3. 15.]
```

```
print(df.head())
```

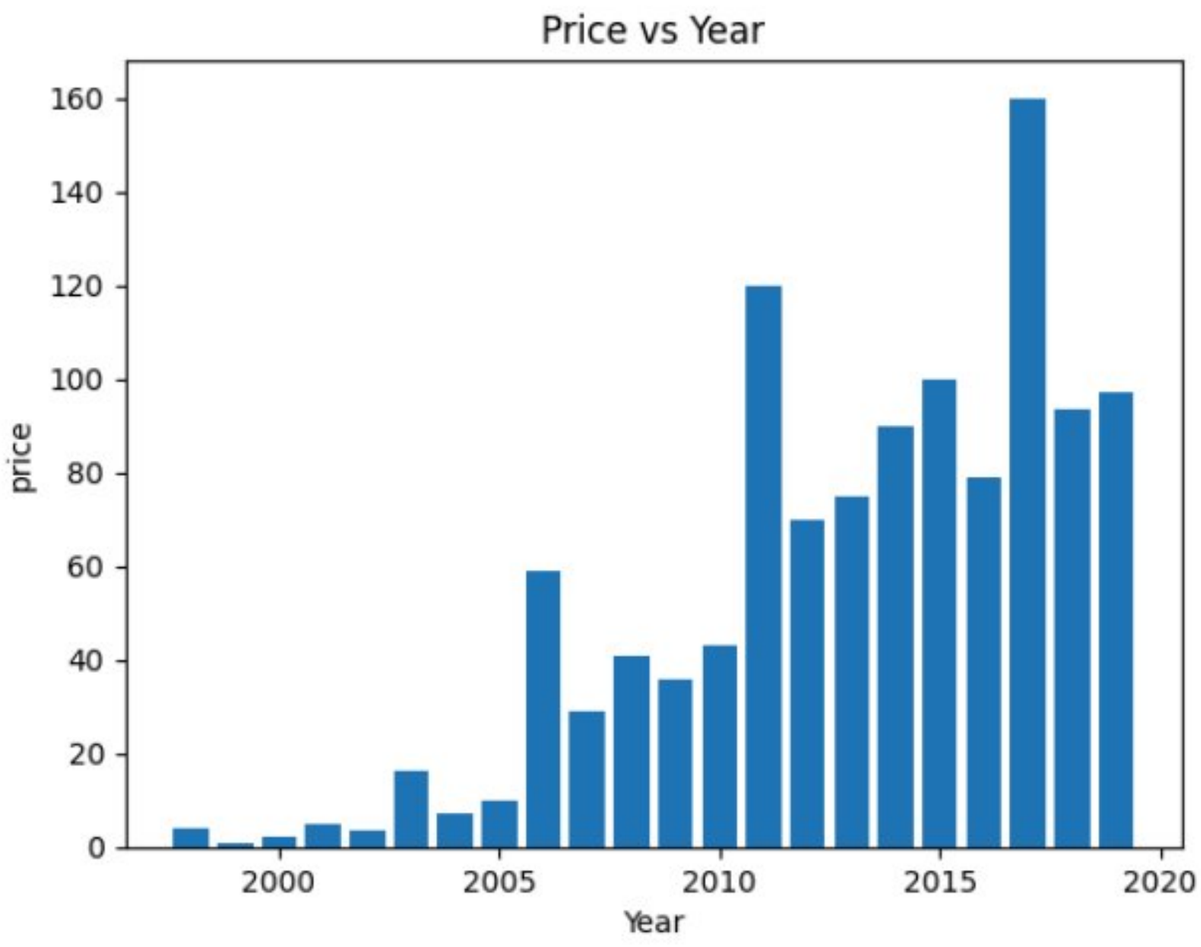
```
   Location  Year  Kilometers_Driven  Fuel_Type  ...   Power  Seats  Pri
0       9.0  2010              72000        0.0  ...   58.16    5.0   1.
1      10.0  2015              41000        1.0  ...  126.20    5.0  12.
2       2.0  2011              46000        4.0  ...   88.70    5.0   4.
3       2.0  2012              87000        1.0  ...   88.76    7.0   6.
4       3.0  2013              40670        1.0  ...  140.80    5.0  17.

[5 rows x 12 columns]
```

DATA VIZUALIZATION

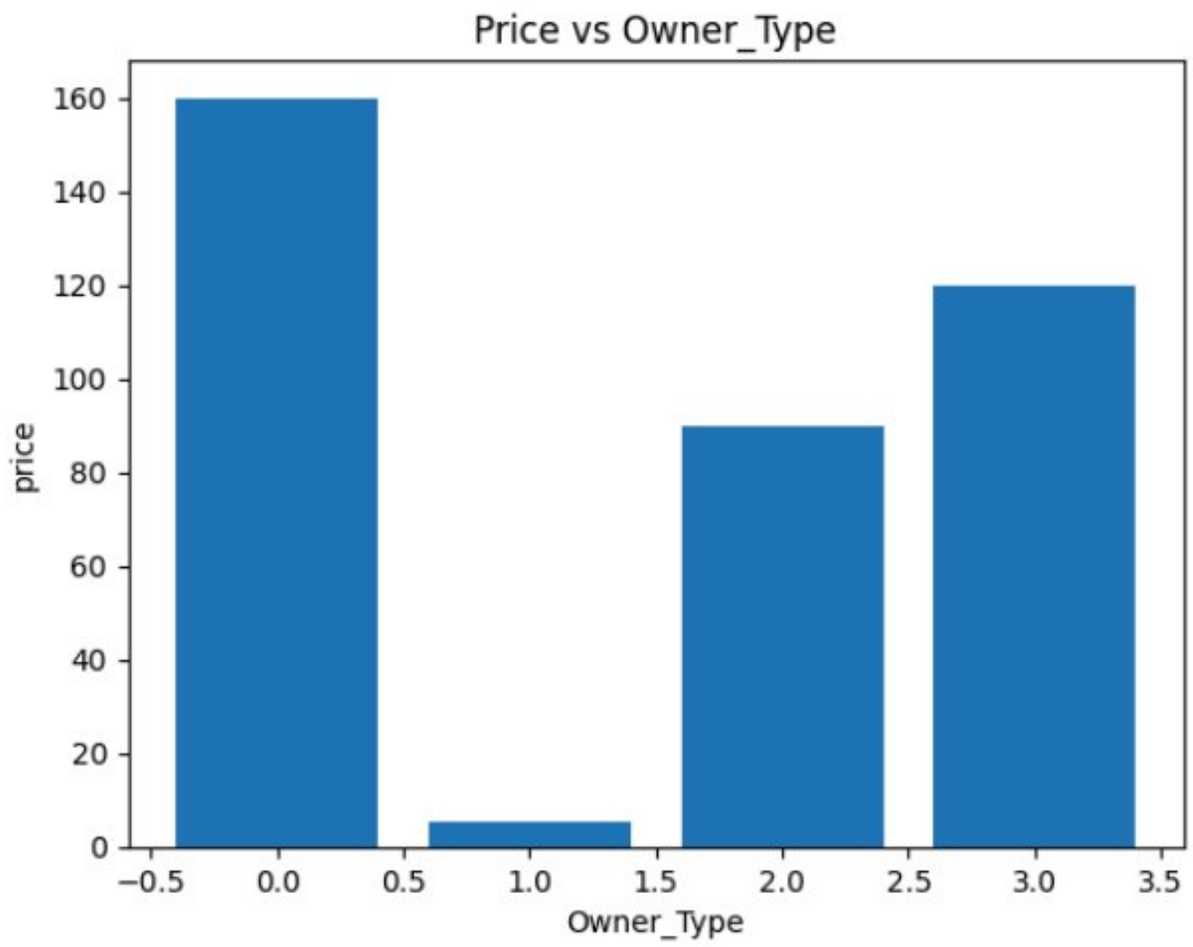
```
#CHECKING THE CORRELATION

plt.figure(figsize=(8,6))
sns.heatmap(df.corr(),annot=True,cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



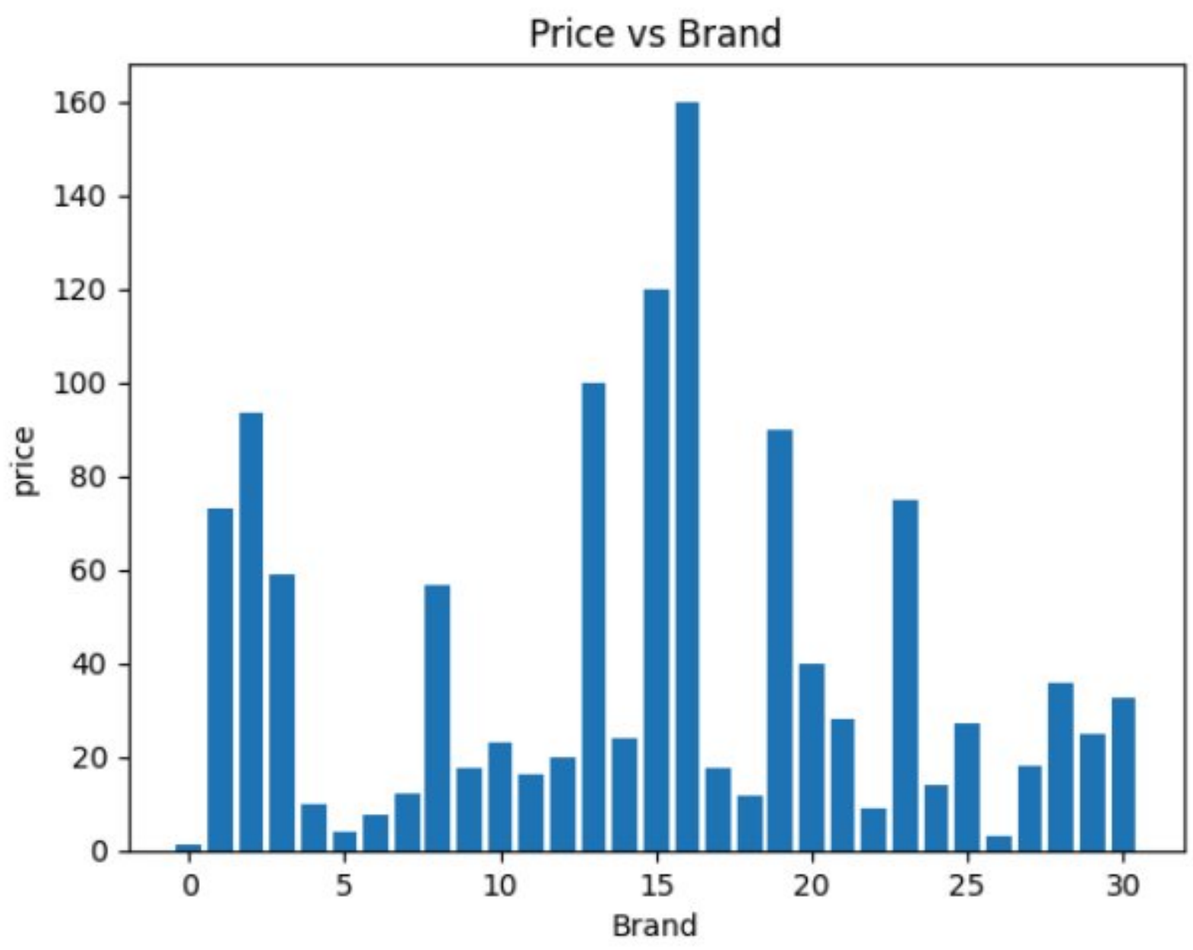

```
#VIZUALIZING PRICE VS YEAR
plt.title('Price vs Year')
plt.xlabel('Year')
plt.ylabel('price')
plt.bar(df.Year,df.Price)
plt.show()
```




```
#VIZUALIZING PRICE VS OWNER_TYPE
plt.title('Price vs Owner_Type')
plt.xlabel('Owner_Type')
plt.ylabel('price')
plt.bar(df.Owner_Type,df.Price)
plt.show()
```




```
#VIZUALIZING PRICE VS BRAND
plt.title('Price vs Brand')
plt.xlabel('Brand')
plt.ylabel('price')
plt.bar(df.Brand,df.Price)
plt.show()
```



CREATING FEATURES AND TARGET

```
#created train and test datasets
X=df.drop(['Price'],axis=1)
y=df['Price']
print("created train an test datasets")
```

```
created train an test datasets
```

```
#splittling the data
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random
print("Created train an test datasets")
```

```
Created train an test datasets
```

CREATING A PIPELINE

```
def model_pipeline(model):
    pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="mean")),
        ("scaler", MinMaxScaler()),
        ("feature_selection", SelectKBest(score_func=f_regression, k=8)
        ("model", model)
    ])
    return pipeline
```

SELECTING THE BEST MODEL

```
#model selection
def select_model(X,y,pipeline_func):
  models={
      "linearregression": LinearRegression(),
      "Decisiontree": DecisionTreeRegressor(),
      "Randomforest": RandomForestRegressor(),
      "XGBRegressor": XGBRegressor(),
      "KNeighborsRegressor": KNeighborsRegressor()
  }
cols=["model","runtime","r2_score"]
new_df=pd.DataFrame(columns=cols)
for name,model in models.items():
  start_time=time.time()
  print("pipeline running on",name)
  pipeline=pipeline_func(model)
  scores=cross_val_score(pipeline,X,y,cv=5,scoring="r2")
  row={
      "model":name,"runtime":time.time()-start_time,"r2_score":scores.
      }
new_df=pd.concat([new_df,pd.DataFrame([row])],ignore_index=True)
new_df=new_df.sort_values(by="r2_score",ascending=False)
return new_df
```

TRAINING THE MODEL

```
model=select_model(X_train,y_train,model_pipeline)
```

```
pipeline running on linearregression
pipeline running on Decisiontree
pipeline running on Randomforest
pipeline running on XGBRegressor
pipeline running on KNeighborsRegressor
```

```
model
```

	model	runtime	r2_score
0	KNeighborsRegressor	0.153997	0.85226

CHOOSING THE BEST MODEL AND TRAINING

```
selected_model=KNeighborsRegressor()
pipeline=model_pipeline(selected_model)
pipeline.fit(X_train,y_train)
```

TESTING THE BEST MODEL

```
y pred=pipeline.predict(X test)
```

```
y pred
```

```
array([ 3.508, 13.84 ,  8.496, ...,  5.81 ,  5.87 ,  2.11 ])
```

EVALUTION METRICES

```
r2_score=r2_score(y_test,y_pred)
MAE=mean_absolute_error(y_test,y_pred)
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
```

```
r2 score
```

```
0.8523606749920601
```

```
MAE
```

```
1.830548172757475
```

```
MSE
```

```
18.168420318936874
```

```
RMSE
```

```
np.float64(4.2624429989076535)
```

REGRESSION REPORT

```
def regression_report(y_test, y_pred):
 print("Regression Report")
 print()
 print("MAE :", mean_absolute_error(y_test, y_pred))
 print("MSE :", mean_squared_error(y_test, y_pred))
 print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
 r2=(y_test, y_pred)
 print("R2 Score:", r2)

# call function
regression_report(y_test, y_pred)
```

```
Regression Report


MAE : 1.830548172757475
MSE : 18.168420318936874
```

```
RMSE: 4.2624429989076535
R2 Score: (2868     5.75
5924    10.08
3764     7.85
4144     2.40
```

```
"" Regression Report

  MAE : 1.830548172757475
  MSE : 18.168420318936874
  RMSE: 4.2624429989076535
  R2 Score: (2868     5.75
  5924    10.08
  3764     7.85
  4144     2.40
  2780     1.60
           ...
  5926     0.55
  4216     4.65
  1351     6.93
  4603     5.38
  5668     1.99
  Name: Price, Length: 1204, dtype: float64, array([ 3.508, 13.84 ,  8.496, ...,  5.81 ,  5.87 ,  2.11 ]))
```

```
]  Start coding or generate with AI.
```